

Report

Assignment 01: Multilingual Tokenization and Language Modeling

Machine Learning for Statistical NLP: Advanced

Elisa Beihofner
University of Gothenburg

1 Part 1: The Tokenizers

I started the assignment by setting up a virtual environment on the server and installing all the necessary packages (such as jupyter and pytorch, etc.). I then registered the virtual environment as a jupyter kernel and connected the server to VS Code so I had a clean, convenient working environment. In a jupyter notebook, I first checked that all the language files existed and the contents of README.txt and metadata.json. Based on the data, I wrote the following predictions for the checkpoint:

Which language do you expect to require the most tokens? For the character tokenizer, each language should have roughly similar token counts (since the characters are balanced in the data). For BPE, I expect Chinese, because it's a character-based language, and one character can already be a word. So two different characters might not occur with each other that much, therefore there might be more single character tokens. It might also not be as morphologically rich as Turkish, or even English in comparison.

How do you expect the larger BPE vocabulary to change the representation of each language? I think that each language will have less tokens in total and per sentence and the number of characters per token will go up (since the larger vocabulary will allow longer sequences to become tokens). I expect that Turkish especially will receive noticeably less tokens since it might originally have had more tokens because it's morphologically rich. I expect a similar outcome for English, but less so. For Chinese, the number of characters per token might be lower than for English and Turkish due to it being a character-based language. But it might lead to tokens being expressions or even short utterances.

What kinds of units do you expect BPE to learn in English, Turkish, and Chinese? For

English, I mostly expect BPE to learn tokens that are words, and maybe some suffixes. For Turkish - a morphologically richer language -, I expect it to learn more affixes and stems, as well as words. For Chinese, I expect it to mostly learn characters, possibly words made up of several characters or even expressions made up of several characters.

Which tokenizer do you expect to work best for language modeling? I think the small BPE tokenizer might be the sweet spot. It will probably create larger meaningful chunks while not putting rare larger tokens at a disadvantage in training by creating too rare tokens.

Then I built the character-level tokenizer by first creating a set of all the individual characters in the training data (9442 in total). I turned the set into a sorted list and added <unk> at index 0 to handle unknown characters, making the total number of characters 9443. After that, I created the character-to-integer and integer-to-character mappings using two dictionaries. I wrote the encoder and decoder functions with <unk> as a fallback and then saved the 9443-item vocabulary for the character-level tokenizer.

For the BPE tokenizers, I first tried to implement the HuggingFace BPE model. I chose the Whitespace pre-tokenizer in order to split punctuation from any words to prevent BPE from learning punctuation as separate tokens that can be merged. However, the HuggingFace BPE tokenizer did not work well for my purposes. I realized that using their *vocab_size* parameter and setting it to 2000 for the small vocabulary BPE tokenizer meant that I had to set *limit_alphabet* to 1000. This means that the majority of the original characters would never even be included in the model. I assumed this would affect Chinese with its many characters disproportionately. Therefore, I decided to switch to the SentencePiece BPE tokenizer implementation.

I chose a vocabulary size of 3000 for the smaller BPE tokenizer and a vocabulary size of 10 000 for

the larger one. I originally tried training the smaller tokenizer with a vocabulary size of 2k. This did not work, however, if I wanted to keep the coverage (SentencePiece implementation) higher. I first tried using a coverage of 0.9995, which is recommended for languages like Chinese. But this required a minimum of 6394 vocabulary items. In order to keep the coverage as high as possible, I decided to increase the vocabulary size by 1000 (and decreased the coverage to 0.9929). 10k vocabulary items then seemed like a reasonable step up from 3k, which worked well with the same coverage. The <unk> ID was again put at index 0.

I trained both BPE tokenizers and then evaluated them on the validation data. I wrote an evaluation function to read each language file and count the number of sentences, characters and tokens as well as the number of unknown characters. This was also used to calculate the average tokens per sentence and the average characters per token. I then used a function to encode each validation language file with each tokenizer and get the results from the evaluation function. These results were presented in a table. Lastly, I looked at three sentences from each language in all three tokenizer conditions.

The character-level tokenizer, predictably, produces the highest number of total tokens for each language. Both BPE tokenizers produce lower numbers of tokens across all three languages compared to that. The 3k BPE tokenizer led to higher numbers of tokens, especially for Chinese (358 234), compared to the 10k BPE tokenizer. Still, Chinese has the highest number of tokens with the 10k tokenizer, too (285 630).

With the character-level tokenizer, each language has the highest number of average tokens per sentence, respectively. For English and Turkish, this number decreases with the 3K BPE tokenizer, and even more so with the 10k one. English always has less tokens per sentence on average than Turkish. For Chinese, that number barely decreases between the character-level (38.55) and the 3k BPE tokenizer (37.55). There is only a significant difference with the 10k BPE tokenizer (29.90). The average character per token increases slightly from 1.0 from the character-level tokenizer to the 3k one - but barely so for Chinese (1.03). For Turkish (3.27) and English (3.24), it increases noticeably with the 10k tokenizer, but again barely any increase for Chinese (1.29). With the character-level tokenizer, English (49) has more unknown tokens than Turkish (15). For both BPE encoders, English has 90 un-

known tokens, and Turkish 102. Chinese has by far the highest number of unknown tokens with every tokenizer, especially the BPE ones. The increase between the character-level tokenizer and the BPE ones is dramatic: from 258 to 6971 unknown tokens for both BPE tokenizers.

For all results see table 1.

After looking at a few examples from each language, here are my insights: For English, the 3k tokenizer split words into smaller pieces like "play" to "p | l | l | a | y | e | d", while the 10k one produced larger pieces like "played" and "two". This matches the increase in average characters per token. "ed" in "played" happens to be a meaningful suffix in English, but not all pieces the tokenizers learned were meaningful.

It is similar for Turkish, with "okullarda" being fragmented into "o | k | l | u | l | l | a | r | d | a" with the 3k tokenizer and into "okul | larda" with the 10k one. "lar" and "da" constitute the plural suffix and a locative suffix, respectively. They were both recognized as tokens in the 3k model. For the 10k model, however, "larda" ended up merged. Contrary to the 3k results, "okul" (meaning "school") was also merged with the 10k tokenizer. Therefore both BPE tokenizers were capable of merging actual linguistic units - but in different ways and seemingly not reliably. It is interesting that the correctly merged "lar" and "da" ended up together as one token in the 10k model.

For Chinese, the 3k tokenizer mostly learned single character units. With the 10k tokenizer, there are more multi-character units like 以上, which is a lexical unit made out of two components, and 大学, whose components have individual meanings but form a different word together. Many of them seem to consist of only two characters though. This matches with the relatively low chars/token number for Chinese, compared to English and Turkish. In the two tokens observed more closely, 10k tokenizer has managed to merge characters that actually occur together in this way in natural language.

The differences in results for each language can probably be explained by their structures. Chinese inherently has more characters than Turkish or English, therefore with the limited vocabulary space in the 3k tokenizer Chinese has to receive a big part of it, leaving less to perform multi-character merges. With the 10k vocabulary, there is more room for character merges for Chinese.

Chinese's character-based structure is likely also the reason for it having so many more unknown

Tokenizer	Lang.	Vocab.	Sent.	Chars.	Tokens	Tok./Sent.	Chars/Tok.	UNK
Character	en	9443	4540	368269	368269	81.12	1.00	49
Character	tr	9443	3735	368383	368383	98.63	1.00	15
Character	zh	9443	9554	368266	368266	38.55	1.00	258
BPE-3k	en	3000	4540	368269	226896	49.98	1.62	90
BPE-3k	tr	3000	3735	368383	245266	65.67	1.50	102
BPE-3k	zh	3000	9554	368266	358234	37.50	1.03	6971
BPE-10k	en	10000	4540	368269	113850	25.08	3.23	90
BPE-10k	tr	10000	3735	368383	112661	30.16	3.27	102
BPE-10k	zh	10000	9554	368266	285630	29.90	1.29	6971

Table 1: Tokenization statistics for character-level and BPE tokenizers across English (en), Turkish (tr), and Chinese (zh). “Vocab.” denotes vocabulary size, “Sent.” the number of sentences, “Tok./Sent.” the average number of tokens per sentence, “Chars/Tok.” the average number of characters per token, and “UNK” the total number of unknown tokens.

characters with the BPE tokenizers. There are a lot of distinct characters, and with the lowered character coverage, rare Chinese characters are not receiving their own vocabulary entries. This might be a limitation of my approach.

An interesting result to investigate further is that for the 3k BPE tokenizer, the tokenization of Chinese remained almost at character-level (1.03 chars/token). While the average number of characters per token increased with the 10k tokenizer (1.29), it stayed noticeably behind the averages for English and Turkish (both roughly 3.25). Even with the 3k BPE tokenizer, English and Turkish had higher char/token numbers (1.63 and 1.50, respectively). Chinese therefore needs more tokens to encode the same number of text. It will thus be interesting to see if these differences in tokenization between the languages will also be reflected in the performance of the language models.

2 Part 2: The Language Models

In Part 2, I first reloaded the saved character-level tokenizer vocabulary and rebuilt the text encoder from part 1, including the <unk> character, in a new jupyter notebook. I also reloaded the saved BPE tokenizers, as well as recreated the encoding wrappers from part 1. Then I wrote a function to create token streams (one continuous list with all the characters for each data file), which also adds *eos* after every sentence, so that the language model can later learn sentence boundaries.

I copied the tokenizer configurations from part 1 and then built the training streams for every tokenizer from balanced.txt. I used balanced.txt because it contains all the training data already concatenated in mixed order. After that, I built the validation streams, using the single language files from the validation directory and concatenating

them into a single stream before tokenizing with each tokenizer.

I built a custom dataset to divide each token stream into token windows with a context length of 128. For each window, the targets for next-token prediction consisted of the same window shifted by one token. I then used the pytorch Dataloader to group the windows into batches of 64 for the training datasets for efficient parallel GPU training. The batches were also shuffled between each epoch. This resulted in input and target tensors of size [64, 128] (64 sequences of 128 tokens each). This process was repeated for the validation data minus the shuffling.

Finally, I built a decoder-only Transformer language model using. It is trained from scratch with causal self-attention. It has 2 Transformer layers with 4 attention heads each. The hidden/embedding dimension is 256, the feed-forward dimension is 1024, the context length 128 and dropout is 0.1. The vocabulary size depends on the tokenizer used beforehand. The model turns each token ID into a 256-dimensional vector, it does the same for each position within the context length (learned token and positional embeddings). The causal attention mask prevents the model from looking at future tokens. In the linear output layer, the 256-dimensional hidden representations are converted into vocabulary scores. The architecture of the Transformer model remained the same over all three tokenizer conditions. But as mentioned, the vocabulary sizes differed and therefore the input embedding and output layer sizes differed too.

The parameter counts of the models were computed afterwards and can be seen in table 2. A larger vocabulary increases the parameters, i.e. the size of the model. It is worth noting that the vocabulary sizes increased by 1 after the addition of the

Model	Vocab.	Total	Embedding	Output
Character	9,444	6,457,060	2,417,664	2,427,108
BPE-3k	3,001	3,151,801	768,256	771,257
BPE-10k	10,001	6,742,801	2,560,256	2,570,257

Table 2: Number of parameters for each model configuration. “Vocab.” denotes vocabulary size, while “Embedding ” and “Output” denote the number of parameters in the embedding and output layers, respectively.

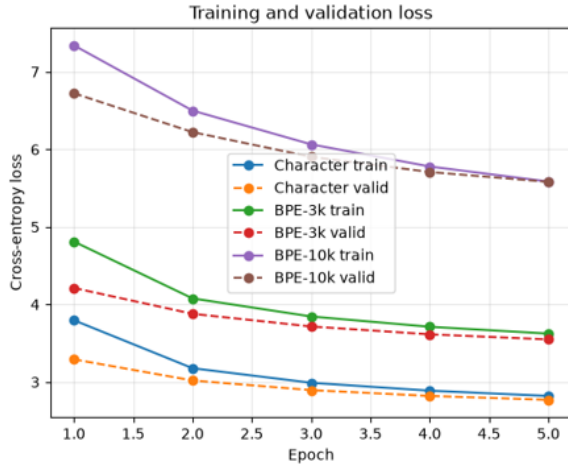


Figure 1: Training and validation loss.

eos ID compared to part 1.

Three models were trained, one for each tokenizer, using a learning rate of $3e-4$ and AdamW optimizer. The models were trained for next-token prediction using cross-entropy loss. Each model was saved after training. The loss for training and validation data was calculated, using one helper function each, recorded and then plotted after training had finished (see figure 1).

As a training-control policy I chose number of complete passes through the corpus (epochs). An advantage of this is that it exposes the models to the same amount of underlying text. However, the tokenizers produced different amounts of tokens, therefore the computational cost per epoch differs between the models. Each model was trained for 5 epochs.

In part 2, I observed the following: The parameter counts show that a larger vocabulary increases the model parameters. For training, the character-level tokenizer needed 8,980,877 tokens to represent the training text. The BPE 3k tokenizer needed 6,780,586 and the BPE 10k tokenizer needed 4,221,411 tokens. The parameters counts for the character model were 6,457,060. They were 3,151,801 for the 3k BPE model and 6,742,801 for

the 10k BPE model.

This is because the model produces an embedding for every possible token and an output score for every possible token. Naturally, the BPE 10k model has the most model parameters. This is interesting because the 10k tokenizer allowed more merges and therefore produced less total tokens than the other two tokenizers. So the 10k language model has the least training tokens but is the largest model. The 3k BPE tokenizer has more training tokens than the 10k one (but less than the character-level tokenizer), but it is the smallest model - less than half the size of the 10k model. This could mean that while a larger BPE vocabulary allows longer sequences and therefore fewer tokens in total, there is a trade-off: the model needs a bigger vocabulary layer to be able to produce a 256-dimensional embedding for every item in the vocabulary and to produce a prediction score for every vocabulary item during next-token prediction.

3 Part 3: Evaluation and Analysis

In a third jupyter notebook I reloaded the character-level tokenizer and the BPE tokenizers. I recreated the encoding wrapper helper functions and the dictionary with the tokenizer conditions. Then I built the test data stream, adding *eos* after every sentence to be consistent with the training data for the LMs. I encoded the test data stream with each of the three tokenizers. This part mirrors the one in the second notebook.

In order to later count bits per character (BPC), I used a function to count each Unicode character in each original test data language file. I also recreated the Transformer model from part 2 and reloaded the trained LMs from part 2, but set them to evaluation mode. After that I calculated BPC by computing the total NLL (negative log likelihood) and then normalizing it with the number of original Unicode characters (excluding newline separators and *eos*) for that language multiplied with the base 2 logarithm. I did this for each model and each token stream in turn, using cross entropy loss sum reduction instead of mean. The results are in table 3.

It is worth mentioning that while *eos* itself isn’t counted as a Unicode character in the denominator, my BPC includes the cost of predicting sentence boundaries, which are part of the corpus structure. It is also to be noted that the Transformer does not receive context from the previous 128-token chunks

Model	English	Turkish	Chinese
BPE-10k	2.284	2.374	6.920
BPE-3k	2.439	2.491	6.850
Character	2.529	2.560	7.039

Table 3: BPC results across English, Turkish, and Chinese for the three tokenizer configurations.

for every new context window. This is consistent with the training setup for all three models, which also used non-overlapping context-windows.

$$\text{BPC} = \frac{-\sum_t \log_2 p(x_t)}{N_{\text{characters}}}, \quad (1)$$

The overall BPC was 3.859 for the 10k BPE tokenizer, 3.927 for the 3k BPE tokenizer, and 4.043 for the character-level tokenizer model, making it the model with the most uncertainty.

For English, BPC decreases from character-level to 3k to 10k (see table 3). As seen in Part 1, when I created the tokenizers, going from a 3k vocabulary to a 10k one caused English (1.62 -> 3.23) and Turkish (1.5 -> 3.27) to have a big increase in chars/token. Here, we can see that BPC also improved between those two (although not quite as dramatically). Using a BPE tokenizer, especially one with a comparably large vocabulary that produces longer sequences, thus seems to lead to a better character-normalized performance and more sure predictions by the LM. Turkish shows the same pattern.

For Chinese, BPC decreases too between the character-level and the 3k tokenizer, but is considerably higher for any tokenizer than for English and Turkish (see table 3). In part 1, chars/token increased between the 3k and the 10k tokenizer chars/token, but less than for English and Turkish (1.03 -> 1.29). And the BPC score for Chinese did not get better, but worse (6.85 -> 6.92) between the 3k and 10k models. Therefore, the 3k BPE model leads to the lowest BPC for Chinese. This is interesting since the 10k model has over double the amount of parameters compared to the 3k model, but still the 3k model has the lowest BPC score for Chinese. The high BPC numbers for Chinese overall, however, indicate that for all three LMs there were a lot more bits required to encode each character, which in turn points to the models being more uncertain when predicting Chinese sequences.

The numbers therefore point to bigger vocabularies that produce longer sequences leading to better performances for English and Turkish, but not for Chinese. Looking back at my checkpoint

report, I expected the LMs that use the 3k BPE tokenizer data to work the best as I assumed that a smaller vocabulary would not lead to unnecessarily rare tokens. This was only true for Chinese, as seen above. I think my original explanation as to why that could be still holds for Chinese, since it is naturally bound to have many rare characters due to its character-based structure. The model is probably already busy trying to predict the "regular" characters of Chinese that are unique parts of the "alphabet" of the data, so that adding additional, rarer token merges might increase its uncertainty. Since the actual number of unique characters for English and Turkish is substantially smaller than for Chinese, it is plausible that having a bigger tokenizer vocabulary that already includes common sequences merged together helps the language models. The models did not have to focus on predicting single characters from a relatively small alphabet, but could also rely on already learned sequences from training.

Another aspect from my checkpoint worth mentioning is that Chinese is disproportionately affected by the lowered coverage for the Sentence-Piece BPE tokenizers. Chinese has the highest amount of unknown characters of the three languages. This might also have led to the high BPC scores for Chinese. Due to the high number of unknowns there is inherently more uncertainty involved in the Chinese tokenized data. This probably carries over to the language models.

Next, I investigated which language gets what part of the vocabulary. I used a function to count how many times a token appeared in the training data for each language. In another function, I added the total counts for each language together and then divided each language count by the total to get the percentages and classify the tokens accordingly. I did this for each BPE tokenizer and then each token in the respective vocabulary in turn. If at least 80% of a token's occurrences were in one language, I counted it as primarily associated with that language. Otherwise, it was counted as shared. The results are in table 4.

For the 3k tokenizer, Chinese receives a significant part of the vocabulary (92.67%), while Turkish and English only receive 1.23% and 0.80%, respectively. 5.30% are shared, no part of the vocabulary is unused. For the 10k tokenizer, Chinese receives the biggest part of the vocabulary again (51.16%), followed by Turkish (18.80%) and English (14.22%). 15.73% of the vocabulary is shared.

Category	BPE-3k	BPE-10k
Chinese	92.67	51.16
English	0.80	14.22
Shared	5.30	15.73
Turkish	1.23	18.80
Unused	0.00	0.09

Table 4: Distribution of BPE vocabulary tokens across language-association categories (%). A token is classified as language-associated when at least 80% of its occurrences in the training data belong to that language; otherwise, it is classified as shared.

This time, however, 0.09% of the vocabulary is unused by any of the three languages. This is especially interesting since the larger vocabulary should allow every part of it to be used.

Intuitively, the results make sense: there is limited room in the 3k vocabulary, and Chinese with its big character inventory likely takes up most of it, leaving little room for the alphabet characters shared by English and Turkish and possible merges - even for Chinese. The 10k vocabulary with its bigger capacity likely allows for accommodating individual Chinese characters plus more English/Turkish merges, as well as some Chinese merges.

Earlier during the checkpoint, I had already observed that in all three tokenizer conditions, the tokenization of Chinese remained (almost) at character-level. This was especially clear in the 3k BPE tokenizer (1.03 chars/token). The results above can now explain why that is the case. My assumption that Chinese takes up most of the room in the shared (3k) vocabulary simply to accommodate its single characters is supported by inspecting the first 1000 most common characters in the Chinese 3k vocabulary. From manually inspecting the list, I could only see single characters, no merged ones. While there were some merged characters in Latin script associated with Chinese, the Chinese characters themselves were not merged. Interestingly, I also found single Chinese characters associated with English and Turkish, when inspecting the most common 500 tokens. They had low occurrence numbers of mostly 1 and 2. But this further proves that Chinese characters received most of the 3k vocabulary capacity if substantial parts of the English and Turkish associated tokens are actually Chinese characters. This then supports my hypothesis that Chinese tokenization stayed so close to character-level out of necessity to accommodate its own vocabulary size.

Additionally, I also excluded the <unk> character from a subsequent vocabulary distribution analysis since it is technically not a character. This barely affected the results though. The most notably change was for the Chinese-associated vocabulary in the 3k condition. Even though <unk> was the fourth common character for Chinese, its 54,480 occurrences associated with Chinese only constitute around 0.018% of the total Chinese characters in the training data.

The fact that Chinese was compressed less strongly than English and Turkish by the BPE tokenizers (i.e. less multicharacter / subword merges) in combination with the tokenization for Chinese being more compact with the 10k BPE tokenizer is also an interesting pattern for relating tokenization to model behavior. In the checkpoint I stated that I wanted to see how this affects the performance of the language models.

The two aspects mentioned above are reflected in the average character per token scores for Chinese in part 1: 1.03 for the 3k tokenizer and 1.29 for the 10k one - a low char/token score that increases with the larger tokenizer vocabulary. For the test data, the number of tokens in Chinese goes down from 367,875 to 295,479 between the 3k and the 10k BPE tokenizers, also showing that more merges are performed. The examples sentences that I looked at in part 1 also reflect this change, with there being mostly single character tokens for 3k but a few multi-character tokens for 10k (see Part 1). The drastic change in the vocabulary allocation from 92.67% of the 3k vocabulary being associated with Chinese to 51.16% of the 10k vocabulary points to more Chinese merges and a more compact tokenization with the large vocabulary, too. But while the Chinese tokenization was more compact with the 10k BPE tokenizer, this ultimately did not improve the Chinese test BPC for the 10k model (6.92 for 10k and 6.85 for 3k). This is interesting because the data and results for English and Turkish might lead to the assumption that a larger BPE vocabulary that allows more merges and therefore longer tokens makes for a better LM in the end.

Looking at the language models shows that the 10k LM for Chinese had more than double the parameters than the 3k one (6.74 million vs. 3.15 million). This is probably mainly because the larger vocabulary requires more embedding and output layers and does not necessarily point to a stronger model. So having a bigger vocabulary with more compact content that also allows the model to learn

multi-character tokens might not always be an advantage. The percentages for the vocabulary allocation show that the model simply has to learn more distinct tokens with the 10k model (< 3000 tokens for the 3k model, but > 5000 for the 10k model). Many of these additional tokens might be relatively rare and therefore hard to learn, leading to more uncertainty. The relatively high number of Chinese associated tokens in combination with less frequent tokens as learning challenges for the LMs could be possible explanations for the worse BPC with the 10k model.

So far, I have thoroughly investigated how Chinese is represented in the different BPE tokenizer conditions and how that affects the model performances. However, another interesting pattern from my checkpoint results is that English and Turkish seem to behave similarly (when compared with Chinese) in terms of token counts and average characters per token. In the tokenizer statistics from Part 1, the total number of tokens decreased from 368,269 (character-level) to 226,896 (3k BPE) to 113,850 (10k BPE) for English and from 368,383 to 245,266 to 112,661 for Turkish. The average chars/token for English increased from 1.0 (character-level) to 1.62 (3k BPE) to 3.23 (10k BPE). For Turkish it increased from 1.0 to 1.50 to 3.27. Turkish was a little more compressed than English with the 10k BPE tokenizer, while English was more compact than Turkish with the 3k BPE tokenizer. Overall, they showed rather similar behavior.

Having seen the final results for both languages, the trend continues with the 3k model BPC scores: 2.44 for English and 2.49 for Turkish. With the 10k one it is 2.28 for English and 2.37 for Turkish. But when examining the vocabulary allocation, Turkish seems to receive a bigger part of the vocabulary than English for both the 3k and the 10k condition: 1.23% for Turkish vs. 0.80% for English (3k) and 18.80% vs. 14.22%.

While the 3k BPC scores are relatively close for both languages (but a little bit better for English), the BPC for English is noticeably better than for Turkish for the 10k model. This is surprising since Turkish had a slightly higher char/token number with the 10k BPE tokenizer than English. But in combination with the data from the vocabulary allocation, I suspect that the reason might be similar to my theory as to why the Chinese 3k model performed better than the 10k model. While Turkish is tokenized more compactly with the 10k vocabulary, it is also consistently assigned a bigger part

of the shared vocabulary than English. With that presumably comes more learning to be done for the language models and possibly more uncertainty, thus higher BPC scores.

In order to see how the tokenization connects to the results, I chose to look at some test sentences for both languages. For the analysis, I excluded sentences that were longer than the context window of 128 tokens. This led to Turkish losing more sentences than English (165 vs. 61 lost). For the rest of the sentences, the BPC with either BPE model was calculated independently for each sentence. I determined whether a sentence had better BPC with the 3k or the 10k model by subtracting the 3k BPC from the 10k BPC. If the resulting number was positive, 3k had lower BPC, if it was negative, 10k had lower BPC.

The best-performing Turkish sentences for the 10k model generally were shorter than the best ones for the 3k model, for example: "Çepni kültürü vardır.", 8 tokens (19 with the 3k model), was the sentence for 10k with the biggest advantage over 3k. But the fourth best-ranking sentence for the 3k model was "Star Wars'un ilk taslağında "Cos Dashit" olarak adlandırılmıştı." with 44 tokens (24 for the 10k model). English shows a similar pattern.

The tokenization of "Çepni kültürü vardır." was ['_', 'Ç', 'e', 'p', 'n', 'i', '_k', 'ü', 'l', 't', 'ü', 'r', 'ü', '_v', 'ar', 'd', 'ır', '.'] for 3k with a BPC of 2.77, and ['_Ç', 'ep', 'ni', '_kültür', 'ü', '_vardır', '.'] for 10k with a BPC of 1.78 (with the underscores representing whitespace). This example shows nicely that the 3k model stayed mostly at character level, while the 10k model predicted bigger chunks. Those bigger chunks could be more frequent and therefore easier to predict, which is why the sentence performed better with the 10k model.

The other example sentence was tokenized as ['_S', 't', 'ar', '_ ', 'W', 'ar', 's', '"', 'un', '_ ', 'il', 'k', '_t', 'as', 'l', 'a', 'ğ', 'ın', 'da', '_ ', '"', 'C', 'o', 's', '_D', 'as', 'h', 'it', '"', '_o', 'lar', 'ak', '_a', 'd', 'l', 'and', 'ıl', 'ır', 'm', 'i', 'şt', 'i', '.'] for 3k and as ['_St', 'ar', '_W', 'ars', '"', 'un', '_ilk', '_tas', 'la', 'ğında', '_ ', 'C', 'os', '_D', 'ash', 'it', '"', '_olarak', '_ad', 'land', 'ılır', 'mıştı', '.'] for 10k. Again, the 3k model tokenization has smaller chunks. The 10k version has bigger chunks, but also a few smaller pieces with less than 3 characters. The BPC score for this sentence was 2.63 at 3k and 3.18 at 10k. This shows that despite the more compact 10k tokenization, the model assigned a

lower probability to this sentence. This is in contrast to the first example, and shows that a more compact tokenization does not automatically make a sentence easier to predict.

English shows a similar pattern, with compact tokenization not being a reliable predictor for a lower BPC score. Two examples are: "Elections are also held." with a BPC of 2.08 and 18 tokens for the 3k model, and a BPC of 1.01 and 8 tokens for the 10k model. "Higher temperatures are needed." with a BPC of 1.72 and 20 tokens for the 3k model, and a BPC of 2.12 and 10 tokens for the 10k model. For both sentences the 10k version uses bigger chunks, but the BPC scores differ.

Another interesting observation is that many of the sentences that have the biggest BPC difference with the 10k model and perform better with the 3k model include uncommon names or sentences structures in English such as: "Swanhilda", "Coppélia", "Image:Excetuion Chamber at [...]", "Thirunelveli" etc. The top 10k sentences have more conventional structures and content like "They are expensive too" or "Andrew asked Leonard to join him".

The 10k tokenizer consistently produces more compact representations, but the sentence examples show that compactness alone does not predict which model obtains lower BPC. For Çepni kültürü vardır., the larger 10k pieces coincide with substantially better performance, whereas the Star Wars sentence is represented with almost half as many tokens by 10k but receives worse BPC. This suggests that the usefulness of larger BPE units depends not only on sequence length but also on how well those particular units and their contexts were learned from the training corpus. The Turkish 3k sentences also include non-Turkish names like "Bacillus Calmette-Guérin", "Star Wars" and "Cos Dashit". Those foreign names remain close to character-level even with the 10k model: ['_St', 'ar', '_W', 'ars'], ['C', 'os', '_D', 'ash', 'it'], ['_Sw', 'an', 'h', 'il', 'da'], ['_C', 'op', 'p', 'é', 'li', 'a'], ['_Th', 'ir', 'un', 'el', 'v', 'eli']. Intuitively this makes sense, because foreign names are probably rare and make it hard for them to be learned by a model.

To summarize, while the 10k model ultimately does lead to lower BPC scores overall for both languages, the assumption that the 10k model performs better for every sentence is not true. The length of a sentence could potentially play a role in its performance, but it also seems to matter which

pieces a tokenizer creates and how the model learns them and which of them a sentence contains.

4 Conclusion

Looking back at my checkpoint, my expectation that the average number of characters per token would stay low was supported, as was the expectation that Chinese would stay relatively close to character-level in all tokenizer conditions (see table 1). I was suprised to see, however, that the 3k BPE language model did not perform best for all three languages. I initially thought that allowing more rare tokens with the bigger 10k vocabulary might lead to a worse LM performance, but this was not the case for English and Turkish. Only Chinese had the lowest BPC score with the 3k model (see table 3).

I think my experiment showed that using a shared tokenizer for English, Turkish, and Chinese does not benefit all languages equally. Chinese's large character vocabulary took up a big part of the shared 3k and 10k BPE tokenizer vocabularies (see table 4). Especially in the 3k condition, this prevented token merges for Chinese, but it also prevented the tokenizer from learning bigger chunks for English and Turkish. While the bigger 10k vocabulary allowed for more compact tokenizations for English and Turkish, there were still less multi-character merges for Chinese (see table 1). Overall, the shared vocabulary was an interesting experiment to explore the differences between these languages, but it comes with limitations.

I would ultimately choose to use the 10k BPE tokenizer for this multilingual setting. Both BPE models performed better than character-level baseline, but the bigger 10k vocabulary allowed for more character merges for all three languages. And while the increased vocabulary allocation and more compact tokenization did not automatically translate into the best Chinese LM performance with this tokenizer, it worked well for English and Turkish and still reasonably well for Chinese. The overall BPC was also lowest for the 10k LM with a score of 3.86. I would say that choosing the largest vocabulary gives each language the fairest chance at taking up space in the vocabulary.